
PORTFOLIO

Movie Reservation Infrastructure Project

Linux / Cloud / System Engineer

작성자 : 오택현	oth93677971@gmail.com
-----------	-----------------------

목차

- 01 프로젝트 개요
- 02 프로젝트 목표
- 03 아키텍처
- 04 VMware 3-Tier 구축
- 05 AWS 3-Tier 구축
- 06 운영 자동화
- 07 Kubernetes

목차

08 장애 해결 사례

09 회고

1. 프로젝트 개요

React 기반 영화 예매 서비스를 Web / WAS / DB로 분리하여 3-Tier 아키텍처를 구성하고, VMware 온프레미스 환경에서 구축한 서비스를 AWS 클라우드 환경으로 확장했습니다. 이후 서버 상태 점검 자동화와 Kubernetes 컨테이너 운영까지 단계적으로 적용하여 서비스 구축뿐 아니라 운영 관점의 인프라 경험을 목표로 진행했습니다.

주요기능: 영화 목록 조회, 좌석 조회 및 선택, 예매 등록, 예매 내역 저장, API Health Check

기술 스택:

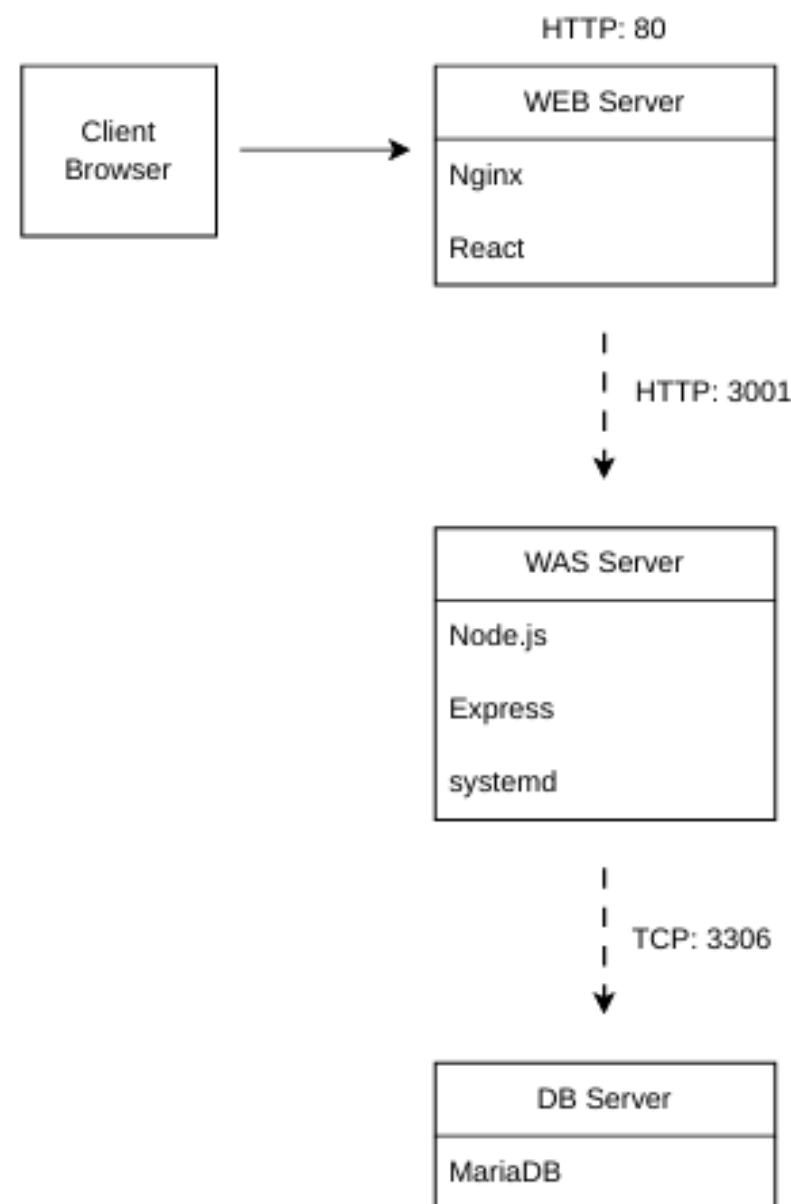
- Frontend: React / TypeScript / Vite
- Backend: Node.js / Express
- Database: MariaDB
- Infrastructure: Rocky Linux / VMware / AWS / Nginx / systemd / Shell Script / Kubernetes

2. 프로젝트 목표

하나의 애플리케이션을 Web / WAS / DB 서버로 분리하여 3-Tier 아키텍처를 구성하고, 서버 간 통신 및 서비스 운영 흐름을 이해하는 것을 목표로 했습니다.

- ✓ Web / WAS / DB 역할 분리 및 3-Tier 구조 이해
- ✓ 서버 간 네트워크 통신 구조 이해
- ✓ Linux 기반 서비스 구축 및 운영
- ✓ 온프레미스 환경에서 AWS 클라우드 환경으로 확장
- ✓ 반복적인 서버 상태 점검 자동화
- ✓ 컨테이너 기반 서비스 운영 경험

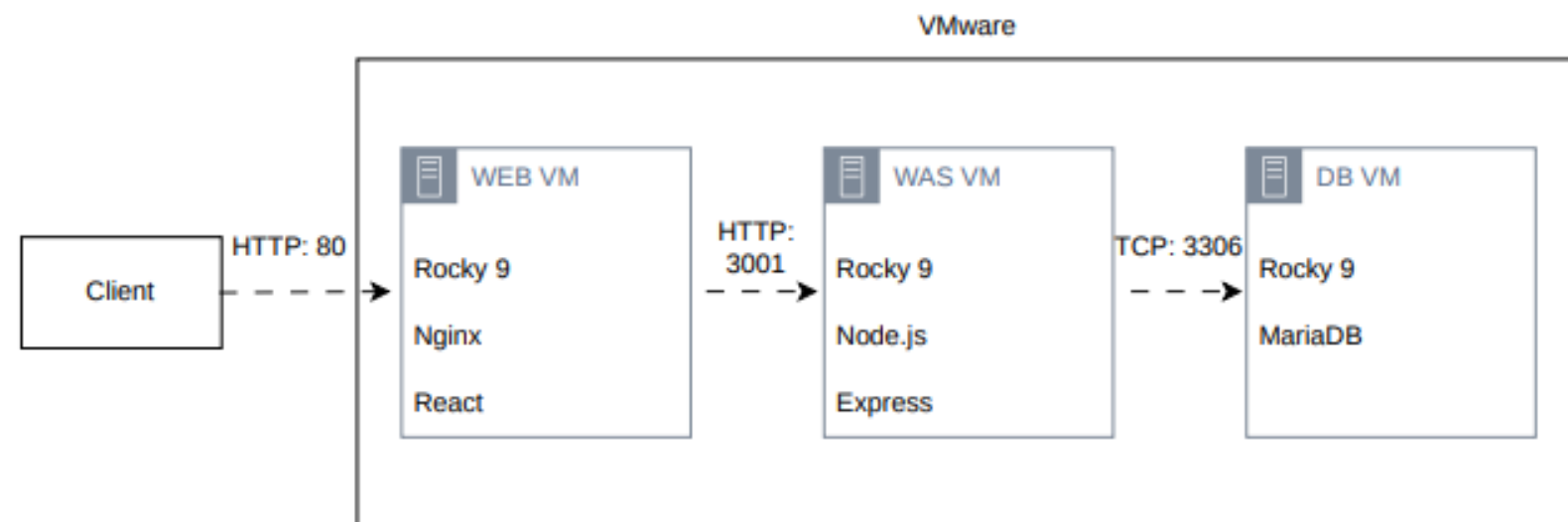
3. 전체 서비스 아키텍처



전체 서비스 아키텍처

- 서비스 전체 구조를 Web / WAS / DB로 분리한 3-Tier 아키텍처로 구성
- Client는 HTTP 80을 통해 Web Server의 Nginx에 접속
- Nginx는 React 정적 파일을 제공하고, API 요청을 Reverse Proxy를 통해 WAS의 3001 포트로 전달
- WAS의 Node.js / Express는 비즈니스 로직을 처리하고 MariaDB와 TCP 3306으로 통신하여 데이터를 조회 및 저장
- 각 계층의 역할을 분리하여 Client → Web → WAS → DB로 이어지는 서비스 요청 및 데이터 처리 흐름을 구현

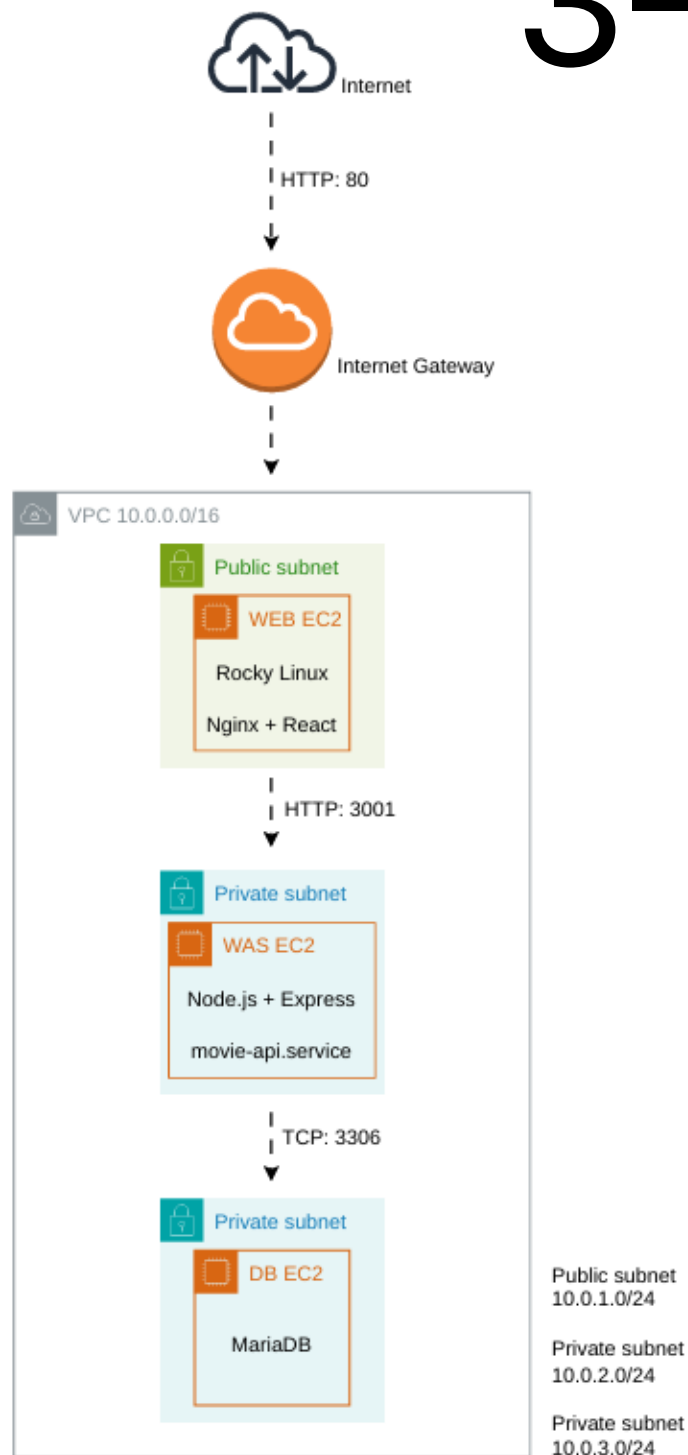
3-2. VMware 아키텍처



VMware 아키텍처

- VMware 환경에 Web / WAS / DB 서버를 각각 Rocky Linux 9 VM으로 구성
- Client는 HTTP 80을 통해 Web VM의 Nginx에 접속
- Nginx는 React 정적 파일을 제공하고, API 요청을 Reverse Proxy를 통해 WAS의 3001 포트로 전달
- WAS의 Node.js / Express는 비즈니스 로직을 처리하고 DB VM의 MariaDB와 TCP 3306으로 통신
- 서버별 역할을 분리하여 3-Tier 구조와 서버 간 통신 흐름을 직접 구현

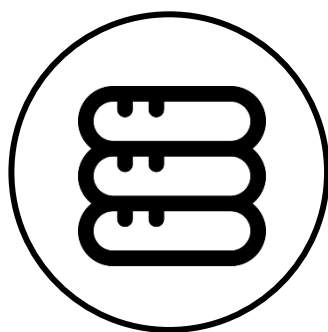
3-3. AWS 아키텍처



AWS 아키텍처

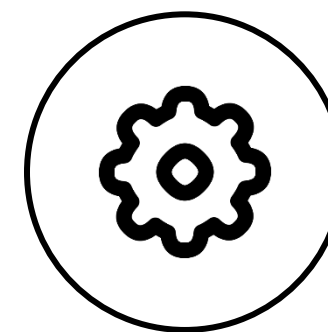
- AWS VPC 10.0.0.0/16 내부에 Web / WAS / DB 서버를 각각 EC2로 구성
- Web EC2는 Public Subnet에 배치하고 Internet Gateway를 통해 외부의 HTTP 80 요청을 수신
- Web EC2의 Nginx는 React 정적 파일을 제공하고, API 요청을 Reverse Proxy를 통해 Private Subnet의 WAS EC2 3001 포트로 전달
- WAS EC2의 Node.js / Express는 비즈니스 로직을 처리하고, Private Subnet의 DB EC2 MariaDB와 TCP 3306으로 통신
- WAS와 DB를 Private Subnet에 분리 배치하여 외부 직접 접근을 제한하고, AWS 환경에서도 3-Tier 구조와 계층별 네트워크 분리를 구현

4. VMware 3-Tier 구축



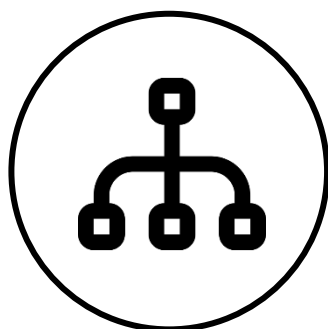
구축 개요

Web / WAS / DB를 각각 독립된 VM으로 구성하여 서버 역할을 분리하고, 계층 간 네트워크 통신을 직접 구현했습니다.



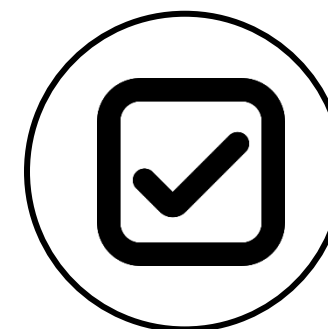
구축 환경

- Hypervisor : VMware
- OS : Rocky Linux 9
- WEB : Nginx + React
- WAS : Node.js + Express
- DB : MariaDB



통신 구조

- Client → WEB : HTTP 80
- WEB → WAS : HTTP 3001
- WAS → DB : TCP 3306



구축 결과

- Web / WAS / DB 역할별 VM 분리 구성
- HTTP 80, HTTP 3001, TCP 3306 통신 확인
- Nginx Reverse Proxy를 통한 API 연동 구현
- MariaDB 데이터 조회 및 예매 데이터 저장 확인

4-2. Web / WAS / DB 서버 구성

WEB Server

주요 작업:

- React Build 파일 배포
- Nginx HTTP 80 서비스
- /api Reverse Proxy 구성

DB Server

주요 작업:

- MariaDB 설치 및 서비스 구성
- 영화 / 예매 데이터 저장
- WAS 서버에서만 TCP 3306 접근

WAS Server

주요 작업:

- Express API 서버 구성
- 3001 포트 서비스
- movie-api.service 등록

Reverse Proxy 구성

```
[web01@web01 ~]$ grep -n "location /api" -A 8 /etc/nginx/conf.d/movie.conf
12:     location /api/ {
13-         proxy_pass http://192.168.56.20:3001;
14-         proxy_set_header Host $host;
15-         proxy_set_header X-Real-IP $remote_addr;
16-         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
17-     }
18- }
```

movie-api Active

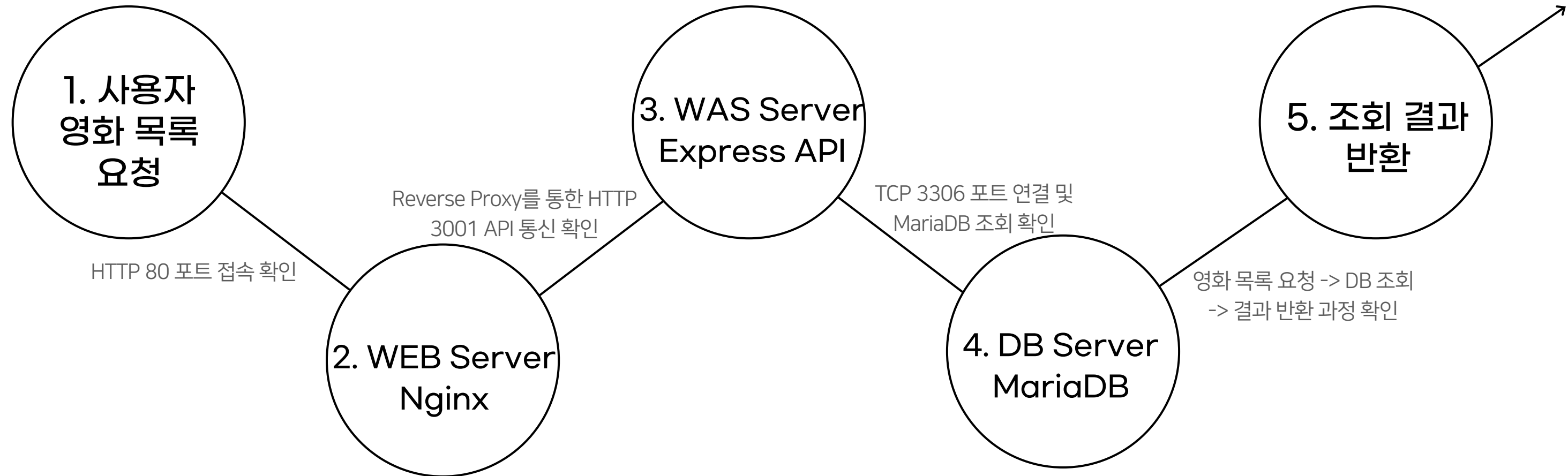
```
[was01@was01 ~]$ systemctl status movie-api
● movie-api.service - Movie Reservation API Server
   Loaded: loaded (/etc/systemd/system/movie-api.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-22 20:57:50 KST; 10min ago
```

```
MariaDB [movie_reservation]> select * from reservations;
+----+-----+-----+-----+-----+
| id | movie_id | seats | customer_name | created_at |
+----+-----+-----+-----+-----+
| 1  | 2       | ["A1"] | oth          | 2026-06-25 07:14:59 |
| 3  | 1       | ["B5"] | teakhyeon    | 2026-07-22 21:18:55 |
| 5  | 5       | ["D5"] | 오택현       | 2026-07-22 21:22:09 |
+----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

MariaDB [movie_reservation]>
```

MariaDB 데이터 조회

4-3. 서비스 통신 검증



☑ 각 계층 간 네트워크 통신과 서비스 요청 흐름이 정상적으로 동작하는지 확인

```
[web01@web01 ~]$ curl http://localhost
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>영화 예매</title>
    <script type="module" crossorigin src="/assets/index-y5nGSMjt.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-B2FX-QRf.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

1. WEB 서비스 확인

- React page 응답

```
[web01@web01 ~]$ curl http://localhost/api/health
{"status":"ok","tier":"was","database":"mariadb"}[web01@web01 ~]$
[web01@web01 ~]$ curl http://localhost/api/movies
[{"id":1,"title":"들편 하 이 머 ", "runtime":166, "genre":"SF/액션", "rating":"8.5", "poster_color":"#c45c26", "poster_url":"/posters/dune2.jpg"}, {"id":2, "title":"오픈 하 이 머 ", "runtime":180, "genre":"드라마", "rating":"8.3", "poster_color":"#1a3a5c", "poster_url":"/posters/oppenheimer.jpg"}, {"id":3, "title":"범 죄 도 시 4", "runtime":109, "genre":"액션", "rating":"7.2", "poster_color":"#8b1a1a", "poster_url":"/posters/roundup4.jpg"}, {"id":4, "title":"인사이드 아웃 2", "runtime":96, "genre":"애니메이션", "rating":"8.0", "poster_color":"#4a90a4", "poster_url":"/posters/insideout2.jpg"}, {"id":5, "title":"파묘", "runtime":134, "genre":"미스터리", "rating":"7.8", "poster_color":"#2d4a2d", "poster_url":"/posters/exhuma.jpg"}, {"id":6,
```

2. API 통신 확인

- API 정상 응답

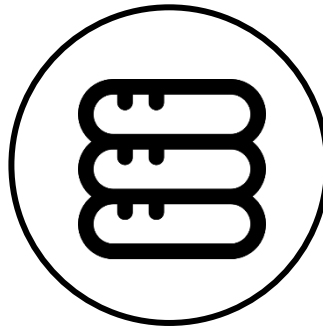
```
MariaDB [movie_reservation]> select * from movies;
+-----+-----+-----+-----+-----+-----+-----+
| id | title           | runtime | genre      | rating | poster_color | poster_url |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 들편 하 이 머   | 166     | SF/액션    | 8.5    | #c45c26      | /posters/dune2.jpg |
| 2 | 오픈 하 이 머   | 180     | 드라마    | 8.3    | #1a3a5c      | /posters/oppenheimer.jpg |
| 3 | 범 죄 도 시 4   | 109     | 액션      | 7.2    | #8b1a1a      | /posters/roundup4.jpg |
| 4 | 인사이드 아웃 2 | 96      | 애니메이션 | 8.0    | #4a90a4      | /posters/insideout2.jpg |
| 5 | 파묘           | 134     | 미스터리   | 7.8    | #2d4a2d      | /posters/exhuma.jpg |
| 6 | 쿵푸팬더 4     | 94      | 애니메이션 | 6.9    | #d4a017      | /posters/kungfupanda4.jpg |
+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.000 sec)
```

3. DB 통신 확인

- 영화 데이터 조회

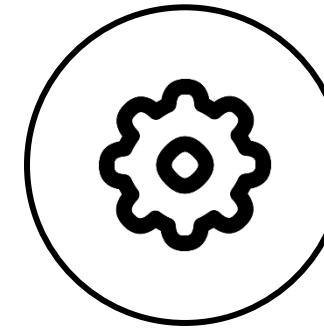
5. AWS 3-Tier 구축

구축 개요



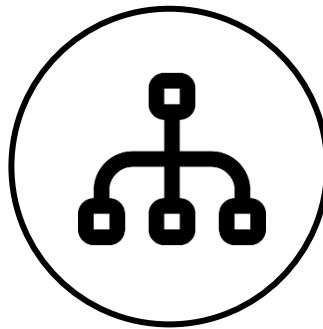
AWS VPC 환경에서 Web / WAS / DB 서버를 각각 EC2로 구성하고, Public / Private Subnet과 Security Group을 활용하여 3-Tier 네트워크 구조를 구현했습니다.

구축 환경



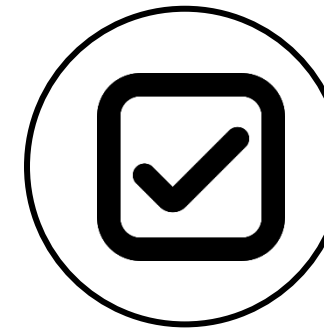
- Platform : AWS
- Compute : EC2
- Network : VPC / Subnet / Security Group
- OS : Rocky Linux 9
- WEB : Nginx + React
- WAS : Node.js + Express
- DB : MariaDB

통신 구조



- Internet → WEB EC2 : HTTP 80
- WEB EC2 → WAS EC2 : HTTP 3001
- WAS EC2 → DB EC2 : TCP 3306
- WEB : Public Subnet
- WAS / DB : Private Subnet

구축 결과



- Web / WAS / DB EC2 역할별 분리 구성
- Public / Private Subnet 기반 네트워크 분리
- Security Group을 통한 계층별 접근 제어
- Nginx Reverse Proxy 기반 API 연동 확인
- MariaDB 예매 데이터 저장 및 조회 확인

5-2. Web / WAS / DB EC2 구성

WEB EC2

- Public Subnet에 배치
- 외부 HTTP 80 요청 처리
- Nginx Reverse Proxy를 통해 Private WAS EC2로 API 전달

WAS EC2

- Private Subnet에 배치
- WEB Security Group에서 오는 3001 요청만 허용
- movie-api.service로 API 서비스 운영

DB EC2

- Private Subnet에 배치
- WAS Security Group에서 오는 TCP 3306만 허용
- MariaDB에 영화/예매 데이터 저장

WEB Subnet / Route Table

subnet-049b2a8458ed22fbb / web-subnet

작업 ▼

세부 정보			
서브넷 ID subnet-049b2a8458ed22fbb	서브넷 ARN arn:aws:ec2:ap-northeast-2:007053125976:subnet/subnet-049b2a8458ed22fbb	상태 Available	퍼블릭 액세스 차단 끄기
IPv4 CIDR 10.0.1.0/24	사용 가능한 IPv4 주소 250	IPv6 CIDR -	IPv6 CIDR 연결 ID -
가용 영역 apne2-az1 (ap-northeast-2a)	네트워크 경계 그룹 ap-northeast-2	VPC vpc-0ceea6be6098f3b11 movie-project	라우팅 테이블 rtb-00fc71afeec79b399 movie-public-route-table
네트워크 ACL acl-08d8c405c0c0eb06c	기본 서브넷 아니요	퍼블릭 IPv4 주소 자동 할당 아니요	IPv6 주소 자동 할당 아니요
고객 소유 IPv4 주소 자동 할당 아니요	고객 소유 IPv4 풀 -	Outpost ID -	IPv4 CIDR 예약 -
IPv6 CIDR 예약 -	IPv6 전용 아니요	호스트 이름 유형 IP 이름	리소스 이름 DNS A 레코드 비활성화됨
리소스 이름 DNS AAAA 레코드 비활성화됨	DNS64 비활성화됨	소유자 007053125976	

플로우 로그 라우팅 테이블 네트워크 ACL CIDR 예약 공유 중 태그 관련 리소스

라우팅 테이블: rtb-00fc71afeec79b399 / movie-public-route-table

라우팅 테이블 편집

라우팅 (2)

라우팅 필터링

대상	대상
10.0.0.0/16	local
0.0.0.0/0	igw-0c9c258b057e99310

✓ 0.0.0.0/0 -> Internet Gateway

WAS Security Group

✓ WEB SG -> TCP 3301 허용

sg-0683ac9490802eb3a - movie-was-sg

세부 정보

보안 그룹 이름
movie-was-sg

소유자
007053125976

보안 그룹 ID
sg-0683ac9490802eb3a

인바운드 규칙 수
3 권한 항목

설명
movie_project_was_sg

아웃바운드 규칙 수
1 권한 항목

VPC ID
vpc-0ceea6be6098f3b11

인바운드 규칙 아웃바운드 규칙 공유 VPC 연결 관련 리소스 -신규 태그

인바운드 규칙 (3)

검색

태그 관리 인바운드 규칙 편집

☐	Name ▾	보안 그룹 규칙 ID ▾	IP 버전 ▾	유형 ▾	프로토콜 ▾	포트 범위 ▾	소스 ▾	설명
☐	-	sgr-07f03df6cb5c44878	-	SSH	TCP	22	sg-028c3183d6095a59...	-
☐	-	sgr-0ac16bfb54ef71fda	-	사용자 지정 TCP	TCP	3001	sg-028c3183d6095a59...	-
☐	-	sgr-07fea58f4a140c392	IPv4	SSH	TCP	22	211.226.32.123/32	-

DB Security Group

✓ WAS SG -> TCP 3306 허용

sg-0f450e02068b75143 - movie-db-sg

세부 정보

보안 그룹 이름
movie-db-sg

소유자
007053125976

보안 그룹 ID
sg-0f450e02068b75143

인바운드 규칙 수
4 권한 항목

설명
movie_project_db_sg

아웃바운드 규칙 수
1 권한 항목

VPC ID
vpc-0ceea6be6098f3b11

인바운드 규칙 아웃바운드 규칙 공유 VPC 연결 관련 리소스 -신규 태그

인바운드 규칙 (4)

검색

태그 관리 인바운드 규칙 편집

☐	Name ▾	보안 그룹 규칙 ID ▾	IP 버전 ▾	유형 ▾	프로토콜 ▾	포트 범위 ▾	소스 ▾	설명
☐	-	sgr-0e6eb5716dd5b7133	-	SSH	TCP	22	sg-028c3183d6095a59...	-
☐	-	sgr-09d9aee6418674be7	IPv4	SSH	TCP	22	211.226.32.123/32	-
☐	-	sgr-0d3ca09549473d7ce	-	SSH	TCP	22	sg-0683ac9490802eb3...	-
☐	-	sgr-0c0df994783bd0636	-	MySQL/Aurora	TCP	3306	sg-0683ac9490802eb3...	-

5-3. AWS Network & Security 구성

WAS EC2 Private Network

☑ Public IPv4 없이 Private Subnet에 배치

DB EC2 Private Network

☑ Private IP 기반 내부 통신 구성

6. 운영 자동화

구분	스크립트	주요 기능	대상	목적
Web 점검	health-check-web.sh	Nginx / HTTP 80 / WAS 3001 / Disk / Memory 확인	Web Server	서비스 상태 점검
WAS 점검	health-check-was.sh	movie-api / API Health / DB 3306 / Disk / Memory 확인	WAS Server	API 및 DB 연결 점검
DB 점검	health-check-db.sh	MariaDB / 3306 / DB 상태 / Disk / Memory 확인	DB Server	DB 서비스 상태 점검
DB 백업	backup-db.sh	MariaDB 데이터 백업	DB Server	데이터 보호
서비스 복구	service-restart.sh	서비스 상태 확인 및 재시작	Server	장애 대응 / 복구

- ✓ Web / WAS / DB 서버의 상태 점검뿐만 아니라 DB 백업 및 서비스 재시작 작업까지 Shell Script로 구성하여 반복적인 운영 작업을 자동화

6-2. 운영 자동화 스크립트 실행 및 검증

```
[ec2-user@ip-10-0-2-144 ~]$ scripts/health-check-was.sh
scripts/health-check-was.sh: line 1: /home/ec2-user/script/health-check-web.sh: No such file or directory
=====
AWS Was Server Health Check
=====

[1] Movie-API Service
[OK] movie-api is running

[2] API Health
[OK] /api/health response success

[3] DB Port Check
[OK] DB 10.0.3.15:3306 reachable

[4] Disk Usage
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1 8.0G  2.0G  6.0G  25% /

[5] Memory Usage
              total        used        free      shared  buff/cache   available
Mem:          912Mi       195Mi       503Mi         0.0Ki       213Mi       584Mi
Swap:           0B           0B           0B

Health check completed.
[ec2-user@ip-10-0-2-144 ~]$
```

health-check-was.sh / service-restart.sh

- 실행 결과

```
ec2-user@ip-10-0-2-144:~$ scripts/service-restart.sh movie-api
Restarting movie-api...
[OK] movie-api restarted successfully
[ec2-user@ip-10-0-2-144 ~]$
```

```
[ec2-user@ip-10-0-3-15 ~]$ scripts/backup-db.sh
[OK] DB backup success: /home/ec2-user/db-backups/movie_reservation_20260802_043439.sql
[ec2-user@ip-10-0-3-15 ~]$ ls -lh ~/db-backups
total 4.0K
-rw-r--r--. 1 ec2-user ec2-user 3.7K Aug  2 04:34 movie_reservation_20260802_043439.sql
[ec2-user@ip-10-0-3-15 ~]$
```

backup-db.sh

- 백업 파일 생성 결과

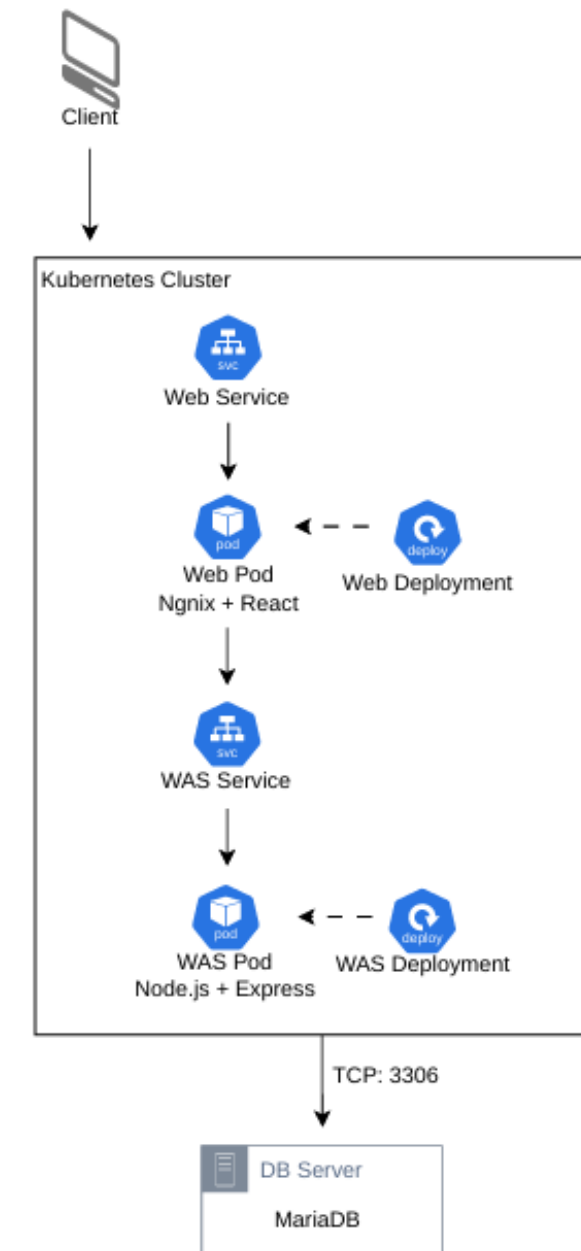
7. Kubernetes

구축 개요

기존 3-Tier 구조 중 Web과 WAS 계층을 Kubernetes 환경으로 컨테이너화하여 배포하고, DB는 기존 MariaDB 서버를 사용하도록 구성했습니다.

구축 환경

- Container : Docker
- Orchestration : Kubernetes
- WEB : Nginx + React
- WAS : Node.js + Express
- DB : MariaDB
- Resource : Deployment / Service / ConfigMap / Secret



7-2. K8s 리소스 구성

구분	리소스	역할
WEB	Deployment / Service	React + Nginx Pod 실행 및 외부 접근
WAS	Deployment / Service	Node.js API Pod 실행 및 WEB 연결
DB	MariaDB	기존 DB 서버를 사용하며 WAS에서 TCP 3306으로 연결
설정	ConfigMap	환경 설정 관리
보안	Secret	DB 계정 / 비밀번호 관리

리소스 상태 확인

- `kubectl get pods`
- `kubectl get svc`
- `kubectl get deployments`

```
k8s01@k8s01:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
movie-was-664f684dcd-dnz99         1/1     Running   11 (20m ago)  10d
movie-web-745694f966-6pkr9         1/1     Running   14 (12m ago)  10d
k8s01@k8s01:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>        443/TCP          12d
movie-was-service    ClusterIP   10.97.210.30 <none>        3001/TCP         10d
movie-web-service    NodePort    10.98.10.199 <none>        80:30080/TCP     10d
k8s01@k8s01:~$ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
movie-was  1/1     1            1           10d
movie-web  1/1     1            1           10d
k8s01@k8s01:~$
```

7-3. 배포 및 통신 검증

- Pod 상태 확인

- kubectl get pods

```
k8s01@k8s01:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
movie-was-664f684dcd-dnz99         1/1     Running   11 (70m ago)  10d
movie-web-745694f966-6pkr9         1/1     Running   14 (62m ago)  10d
[k8s01@k8s01 ~]$
```

- Service 확인

- kubectl get svc

```
k8s01@k8s01:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes          ClusterIP   10.96.0.1    <none>        443/TCP    12d
movie-was-service   ClusterIP   10.97.210.30 <none>        3001/TCP   10d
movie-web-service   NodePort    10.98.10.199 <none>        80:30080/TCP 10d
[k8s01@k8s01 ~]$
```

- Web Pod -> WAS Service API 통신 확인

- kubectl exec <web-pod> -- curl http://<service>/api/health

```
k8s01@k8s01:~$ kubectl exec movie-web-745694f966-6pkr9 -- curl http://movie-was-service:3001/api/health
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    49  100    49    0     0    2027      0  --:--:-- --:--:-- --:--:--   2130
{"status":"ok","tier":"was","database":"mariadb"}
[k8s01@k8s01 ~]$
```

- DB 연결 확인

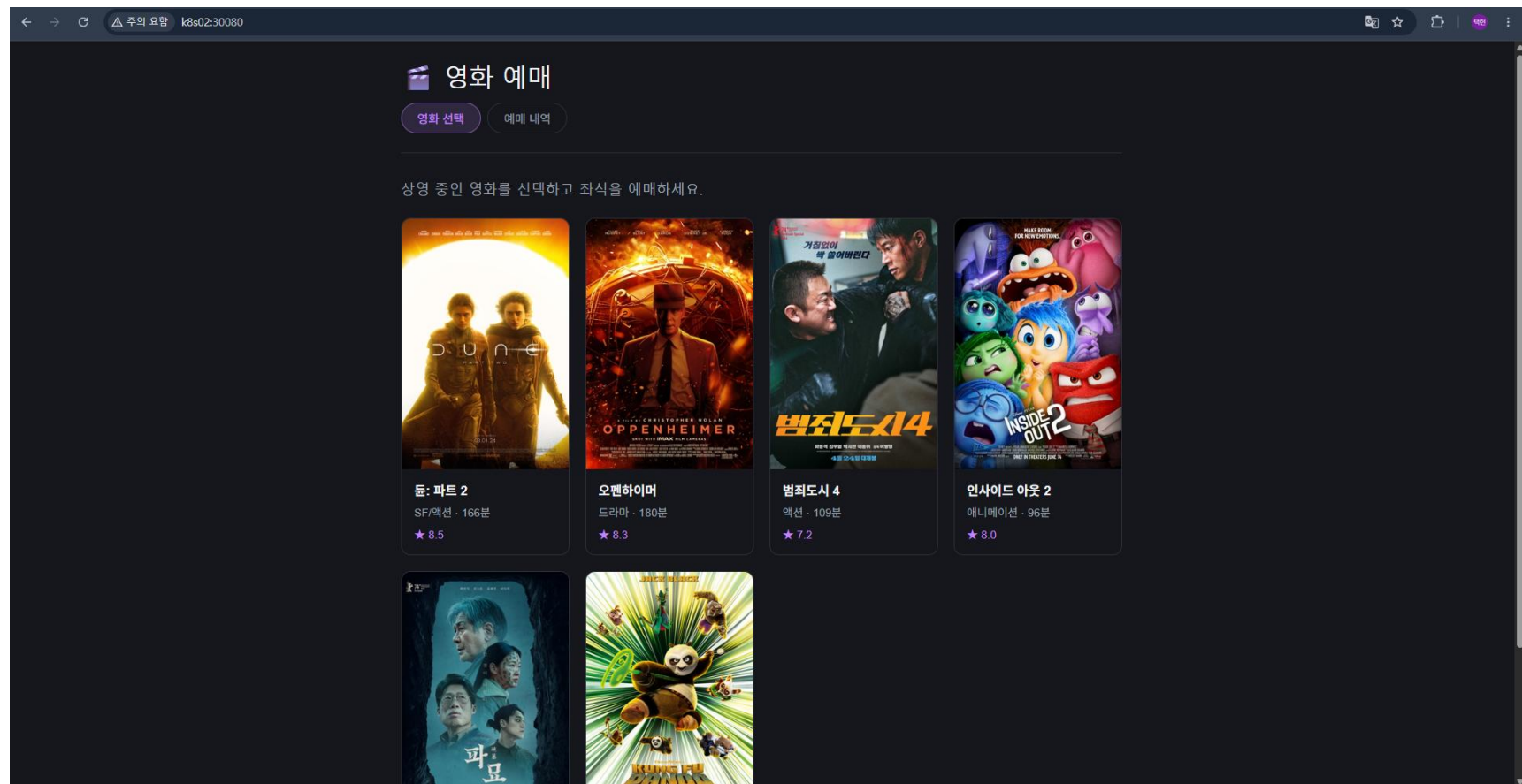
- kubectl exec <was - pod> -- nc -zv 192.168.56.30 3306

```
k8s01@k8s01:~$ kubectl exec movie-was-664f684dcd-dnz99 -- nc -zv 192.168.56.30 3306
192.168.56.30 (192.168.56.30:3306) open
[k8s01@k8s01 ~]$
```

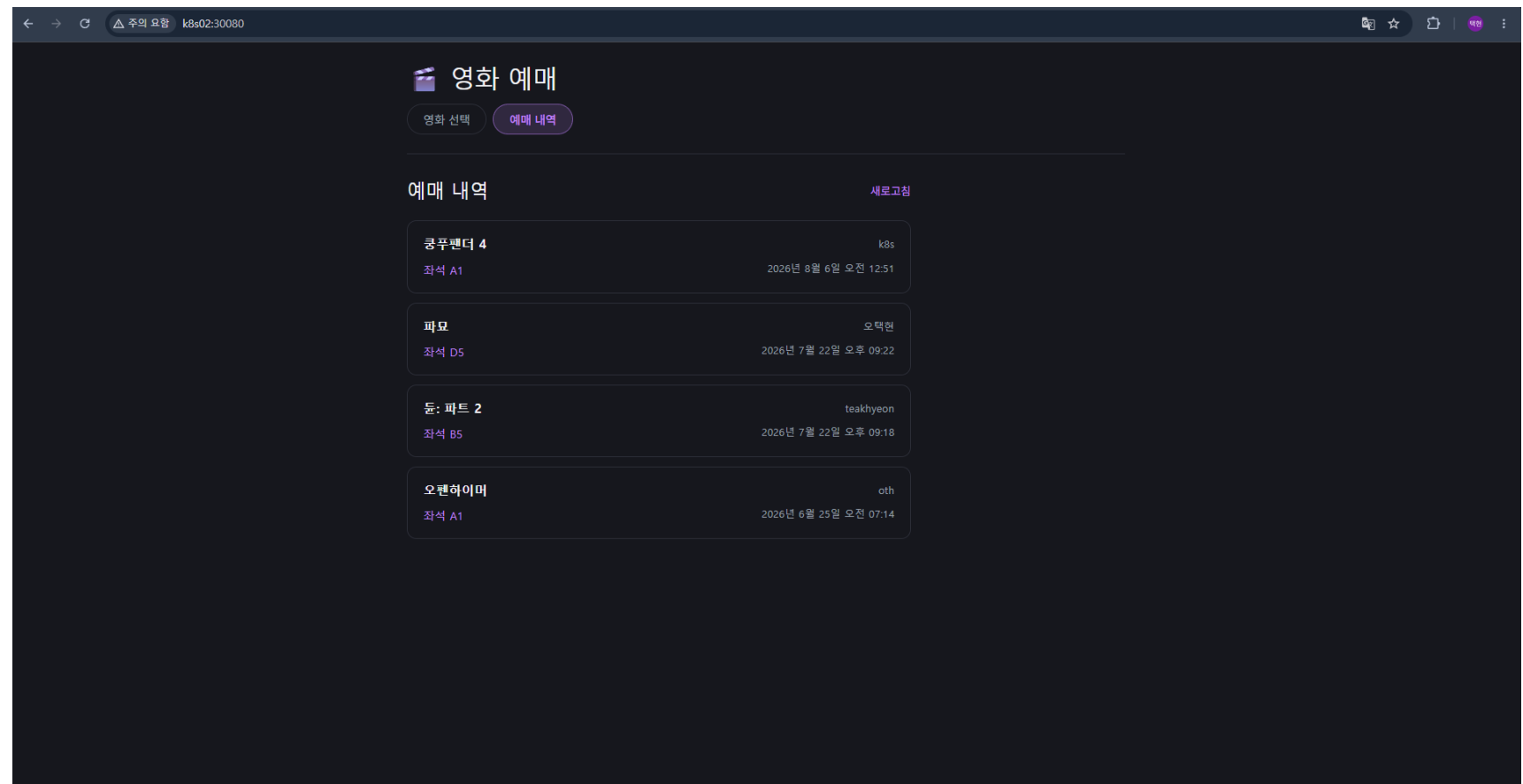

- 서비스 최종 동작 확인

- http://k8s02:30080
- Kubernetes 환경에서 Web / WAS 서비스 배포 후 영화 목록 조회 및 예매 내역 조회가 정상적으로 동작 하는 것을 확인

- 영화 목록 조회 화면



- 예매 내역 조회 화면



8. 장애 해결 사례

Nginx Reverse Proxy 404

- 문제: 브라우저 화면은 정상적으로 출력되었지만 영화 목록이 조회되지 않음
- 원인: Nginx에서 API 요청을 WAS 서버로 전달하는 Reverse Proxy 설정이 누락
- 해결: movie.conf에 /api/ 프록시 설정을 추가하여 WAS 서버의 3001 포트 요청을 전달하도록 구성
- 결과: WEB -> WAS API 통신 정상화

SELinux 502

- 문제: Reverse Proxy 설정 후 404 오류는 해결되었지만 /api 요청에서 502 Bad Gateway가 발생
- 원인: SELinux가 Nginx 프로세스의 외부 네트워크 연결을 제한
- 해결: SELinux 정책을 조정하여 Nginx의 외부 네트워크 연결을 허용
 - `sudo setsebool -P httpd_can_network_connect on`
 - `sudo systemctl restart nginx`
- 결과: Nginx에서 WAS 서버로 정상적으로 연결되어 API 통신 정상화

CNI / Calico

- 문제: k8s02 노드가 NotReady 상태로 표시되고 Pod 네트워크가 정상적으로 구성되지 않음
- 원인: 기존 CNI 설정 파일 충돌
- 해결: 기존 CNI 설정 파일을 정리한 뒤 containerd 와 kubelet을 재시작하고, k8s02의 Calico Pod를 재생성하여 CNI 구성을 초기화
- 결과: Calico Pod가 정상적으로 재생성되고 k8s02 노드가 Ready 상태로 복구됨

DB 접근 권한 문제

- 문제: Kubernetes의 WAS Pod에서 기존 DB 서버의 MariaDB에 정상적으로 접근 불가
- 원인: MariaDB 사용자 계정의 접속 허용 Host가 Kubernetes 노드 / Pod가 사용하는 네트워크 대역과 맞지 않아 접근 권한이 제한됨
- 해결: movieuser 계정이 192.168.56.0/24 대역에서 접속할 수 있도록 MariaDB 사용자 Host와 권한을 구성함
- 결과: WAS Pod에서 DB 서버의 MariaDB로 TCP 3306 통신 및 애플리케이션 DB 연결 정상화

9. 회고

프로젝트를 통해 이해한 것

- 서버 간 통신

동일한 애플리케이션이라도 Web, WAS, DB를 분리하면 각 계층 간 네트워크 통신과 접근 제어가 중요하다는 것을 경험했다.

- 클라우드 네트워크

AWS 환경에서는 서버 구성뿐 아니라 VPC, Subnet, Route Table, Internet Gateway, NAT Gateway, Security Group의 관계를 함께 고려해야 한다는 것을 경험했다.

- 장애 원인 분석

502 오류가 발생했을 때 애플리케이션뿐 아니라 Nginx 설정, 포트 상태, 방화벽, SELinux 등 여러 계층을 순차적으로 점검해야 한다는 점을 학습했다.

- 운영 자동화

서비스 구축 이후 반복되는 운영 점검 역시 중요한 작업이며 이를 Shell Script로 자동화할 수 있다는 점을 학습했다.

감사합니다

010-9367-7971

oth93677971@gmail.com

작성자: 오택현